



**Capstone Final Project – Phase B (61999)**

Real Impact-Factor analysis of academic journals

26-1-R-17

By:

Orian Haziza [orian.haziza@e.braude.ac.il](mailto:orian.haziza@e.braude.ac.il)

Heba Abu Qandil [heba.abu.qandil@e.braude.ac.il](mailto:heba.abu.qandil@e.braude.ac.il)

Supervisors:

Dr. Renata Avros

Prof. Zeev Volkovich

GitHub Link:

<https://github.com/orianhaziza/final-project--impact-factor.git>

## Table of contents

|                                                      |    |
|------------------------------------------------------|----|
| 1. Introduction.....                                 | 2  |
| 2. Mathematical background .....                     | 3  |
| 2.1. Embedding Methods .....                         | 3  |
| 2.1.1. GraphSAGE embedding .....                     | 3  |
| 2.1.2. Node2vec embedding .....                      | 5  |
| 2.2. Link prediction .....                           | 6  |
| 2.3. Suggested algorithm .....                       | 7  |
| 3. Approach .....                                    | 9  |
| 3.1. Rationale and Proposed Solution .....           | 9  |
| 3.2. Impact Factor: Definition and Calculation ..... | 9  |
| 3.3. The Main Algorithm Steps .....                  | 9  |
| 4. Diagrams .....                                    | 11 |
| 4.1. Flow diagram .....                              | 11 |
| 4.2. Sequence diagram .....                          | 12 |
| 5. Testing plan .....                                | 13 |
| 5.1. Result accuracy tests .....                     | 13 |
| 5.2. Formal tests .....                              | 13 |
| 6. Results .....                                     | 15 |
| 7. Conclusion .....                                  | 19 |
| 8. Summary .....                                     | 19 |
| 9. References .....                                  | 20 |

## 1. Introduction

The assessment of academic research quality relies heavily on quantitative metrics, most notably the Journal's **Impact Factor (IF)**.

The IF is a measure reflecting the yearly average number of citations to recent articles published in that journal. It is calculated by dividing the number of citations in the current year to items published in the two previous years by the total number of citable items published in those two years. This metric is not merely a statistic; it **directly influences the perceived prestige** of researchers, journals, and institutions, and serves as a decisive factor in decisions regarding academic promotion, tenure, and research funding.

However, due to its critical importance, the reliability of the IF depends directly on the integrity of the underlying citation network. In recent years, the scientific community has faced a growing challenge of **citation manipulation**, involving practices such as "citation cartels" (groups of researchers reciprocally inflating citations), excessive self-citations, and coercive citations driven by pressure from editors or reviewers [1] [2]. Research (Prabha, 1983) indicates that more than two-thirds of references in a typical paper may be superfluous [6]. These phenomena, along with inherent data noise and recording errors, create distortions that obscure the true scientific impact of journals and articles [3].

The core problem lies in the fact that traditional metric calculations are typically performed on raw data, without filtering out "noise" or validating the reliability of network connections. Existing anomaly detection methods often examine the entire graph statically, potentially missing subtle manipulations or legitimate missing links. Recent studies **suggest a key hypothesis**: genuine citations are robust and steady under network perturbations, whereas manipulative citations tend to be fragile and inconsistent when the network undergoes random structural changes [2] [3].

**In this project**, we propose a novel approach to refining the Impact Factor calculation by constructing a "Reliable Network" from our data. Rather than relying on the collected network as-is, we apply a computational process based on the GraphSAGE algorithm and the Node2Vec algorithm for graph learning. The algorithm aims to achieve a main objective: to filter out suspicious links that are inconsistent with the surrounding scientific fabric and compare the results.

**The primary motivation** is to generate an "**Improved IF**"- a more accurate metric that reflects genuine scientific influence while neutralizing noise and manipulation. We analyze the difference between the regular IF and the improved IF, both at the individual journal level and the system-wide level. This comparison provides a more transparent and complementary tool for research evaluation, highlighting genuine patterns of knowledge while reducing the influence of unreliable data.

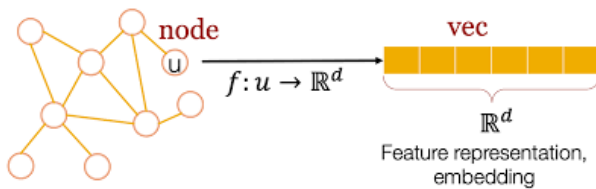
## 2. Mathematical background

The mathematical foundation of the method relies on the assumption that legitimate citations are supported by a stable and coherent network structure, whereas manipulated citations exhibit topological fragility and lack sufficient structural reinforcement[2]. Accordingly, the proposed approach proceeds as follows: a random subset of edges is removed from the citation network, the perturbed graph is embedded into an Euclidean space using the GraphSAGE algorithm and the node2vec algorithm, and the removed edges are subsequently reconstructed through a link prediction procedure. Based on the underlying assumption, genuine citation edges are expected to be consistently recoverable, while manipulated or anomalous edges should repeatedly fail to be reconstructed across perturbations. We will compare the results between the two embedding methods used.

The citation network is treated as an undirected graph, reflecting mutual connectivity between papers and enabling a structural rather than directional analysis. In addition, each node is associated with a corresponding feature vector.

### 2.1 Embedding methods

**Objective:** To map the graph's nodes into Euclidean space as embedding vectors. The assumption is that nodes connected by authentic citations will be positioned close to one another within this space. We will use 2 common embedding methods, in order to get different outlooks on the calculations and compare them.

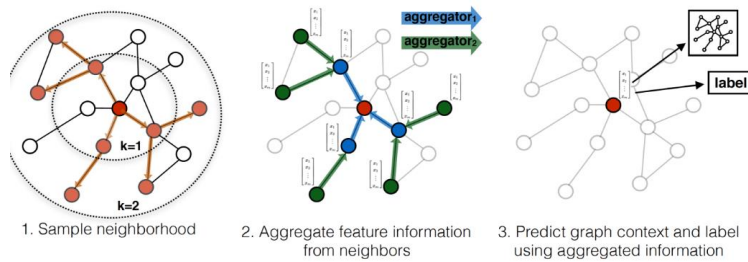


**Figure 1:** Feature representation through an embedding of every node capturing the structure of the underlying network [2]

#### 2.1.1 GraphSAGE embedding-

GraphSAGE (Graph Sample and Aggregate) framework is an inductive learning method designed for generating node embeddings in large and dynamic graphs. Its primary advantage is the ability to generalize to evolving graph structures and to nodes unseen during training [5].

The algorithm produces node representations by sampling and aggregating information from each node's neighbors.



**Figure 2:** Illustration of Sample and aggregate approach [5]

**Sampling-** selecting a fixed-size subset of a node's neighbors

**Aggregating-** combining the sampled neighbors' feature vectors (e.g., by mean or pooling functions) into a single representation that summarizes the node's local neighborhood

### **Algorithm 1 – GraphSAGE Embedding**

#### **Input:**

- A graph  $G = (V, E)$ , where  $V$  is the set of nodes and  $E$  is the set of edges
- Feature matrix  $X$
- Aggregation function
- Number of layers  $K$
- Weight matrices for each layer
- Nonlinear activation function
- Neighborhood sampling size
- Desired embedding dimension

#### **Procedure:**

- 1) Initialization:** For each node  $v$ , the initial representation is its input feature vector.
- 2) Layer-wise propagation for  $k = 1, \dots, K$ :**
  - 2.1) Sample a fixed-size neighborhood for each node
  - 2.2) Aggregate neighbor representations using the chosen aggregation function
  - 2.3) Concatenate the aggregated neighborhood vector with the node's previous representation and apply the corresponding weight matrix
  - 2.4) Apply the nonlinear activation function to obtain the updated representation
- 3) Normalization:** Normalize the resulting embedding to maintain numerical stability.

#### **Output:**

A final embedding vector for each node.

### 2.1.2 Node2vec embedding-

Node2vec algorithm is a method for creating an embedding low-dimensional vector for each node in a graph. This method is meant to capture the graph structural information both locally (nodes in the same area should be close in the embedding space) and globally (nodes with similar role should have similar be close in the embedding space, even if they're far from each other) effectively, while having more control and adaptability in the exploration strategy [4].

This algorithm is inspired by the word2vec algorithm from the natural language processing (NLP) world:

skip-gram word2vec- an algorithm that learns to represent words with similar meaning as vectors close in the vector space by their context in a sentence (meaning, by the words around them)

node2vec- nodes that are well-connected or possess a similar structural role will be close to each other in the vector space. It treats random walks in the graph as “sentences”, and nodes as “words”. Then, the algorithm will use the skip-gram word2vec method to create an embedding for a node as if it's a word in a sentence.

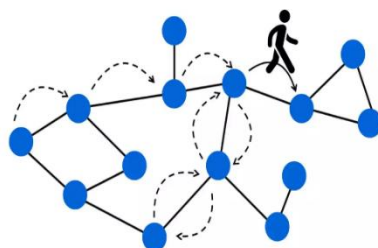
The innovation in this algorithm is that it doesn't walk in the graph randomly, but in a biased way, which balances between the local and global exploration. To balance them, we have two parameters that determine the probability of moving to the next node in the walk:

“return” parameter  $p$ - controls the probability of going back to the node we just came from. High  $p$  value=low probability of going back -> encouraging the walk to go further.

“inout” parameter  $q$ - controls the probability of moving away from the previous node versus staying in the local area. High  $q$  value encourages local exploration (similar to bfs), whereas low  $q$  value encourages exploring further, new areas (similar to dfs).

The probabilities are calculated as  $1/p$ ,  $1/q$ , which enables flexibility in matching the algorithm to the graph structure.

**\*\*disadvantages compared to GraphSAGE:** it can't create an embedding to a new node without training. Sensitive to the choices of  $p$  and  $q$ .



**Figure 3:** Illustration of a random walk [3]

## **Algorithm 2 – node2vec Embedding**

### **Input:**

- A graph  $G = (V, E)$ , where  $V$  is the set of nodes and  $E$  is the set of edges
- “return” parameter  $p$  and “inout” parameter  $q$
- Nwalk- the amount of walks for each node
- Lwalk- the length of each walk
- The final vector dimension  $d$

### **Procedure:**

#### **1) For each node:**

1.1) create Nwalk random walks of length Lwalk each

**2)** All the walks are sent to the skip-gram algorithm, that updates the embedding vectors values of each node

### **Output:**

A final embedding vector for each node.

## **2.2 Algorithm 3 – Link Prediction**

**Objective:** To determine whether an edge should exist between two nodes.

### **Input:**

- Graph  $G$
- The embedding  $W(G)$  of the graph
- A similarity measure
- A similarity threshold

### **Procedure:**

**1)** Compute the similarity score between the embedding vectors of each 2 nodes in the graph using the similarity measure

**2)** Compare the value with the similarity threshold.

- If the similarity exceeds the threshold, the algorithm outputs **1**, indicating that an edge is predicted to exist.
- Otherwise, it outputs **0**, indicating that the nodes are unlikely to be connected.

### **2.3 Suggested Algorithm**

**Objective:** To estimate citation reliability and identify potential citation manipulation within the network, while comparing the results from both embedding algorithms.

**Input:**

- Graph  $G = (V, E)$
- Feature matrix  $X$
- Aggregation function
- Number of layers  $K$
- Embedding dimension
- Weight matrices
- Nonlinear activation function
- Neighborhood sampling size
- Number of perturbations  $N_{iter}$
- Fraction of edges removed per perturbation  $Fr$
- Similarity measure
- Similarity threshold
- “return” parameter  $p$  and “inout” parameter  $q$
- Nwalk- the amount of walks for each node
- Lwalk- the length of each walk



**Procedure:**

- 1) Initialize two arrays Result and Result2 of length  $|E|$  each, filled with zeros.
- 2) For every year in a pre-defined time frame of length T (T iterations):
  - 2.1) For each iteration  $1, \dots, N_{\text{iter}}$ :
    - 2.1.1) Generate a perturbed graph  $G_T$  by randomly removing a fraction  $Fr$  of the edges in  $G$ .
    - 2.1.2) Compute the embedding  $W(G_T)$  using Algorithm 1.
    - 2.1.3) Compute similarity values for all node pairs in  $G_T$  according to  $W(G_T)$ .
    - 2.1.4) Identify the set of reconstructed edges  $ED_R$  using Algorithm 3.
    - 2.1.5) For each edge  $e \in ED_R$ :
 

Update its counter:

$$Result(e) = Result(e) + 1$$
    - 2.1.6) Compute the embedding  $W2(G_T)$  using Algorithm 2.
    - 2.1.7) Compute similarity values for all node pairs in  $G_T$  according to  $W2(G_T)$ .
    - 2.1.8) Identify the set of reconstructed edges  $ED_R$  using Algorithm 3.
    - 2.1.9) For each edge  $e \in ED_R$ :
 

Update its counter:

$$Result2(e) = Result2(e) + 1$$
  - 2.2) Sort the arrays in ascending order.
  - 2.3) For a pre-defined time frame of length T- for every 2 years in the range (T-1 iterations):
    - 2.3.1) calculate regular Impact Factor
    - 2.3.2) calculate improved Impact Factor for the graph with only the reliable edges according to the GraphSAGE embedding model
    - 2.3.3) calculate improved Impact Factor for the graph with only the reliable edges according to the node2vec embedding model
    - 2.3.4) Compare the results

**Output:**

IF score for every 2 years in the range, using the regular method and the improved methods. Edges with low reconstruction scores are interpreted as potentially unreliable or manipulated citations, as they fail to exhibit structural robustness under perturbations. That's why, Using these algorithms, we can compute an improved Impact Factor that discounts unstable or suspicious citation edges. Comparing this refined metric with the standard IF reveals the degree to which manipulated citations inflate journal-level bibliometric indicators.

### 3. Approach

This section describes the methodology developed in this project for detecting manipulations in citation networks and recalculating the Impact Factor (IF) in a more reliable manner. The approach is based on the assumption that legitimate citations are supported by a stable and coherent network structure, whereas manipulated citations exhibit topological fragility and lack sufficient structural reinforcement.

**3.1. Rationale and Proposed Solution-** Our solution focuses on creating a "Reliable Network" from raw data. Instead of relying on the citation network as collected-which may include noise and intentional manipulations (such as citation cartels or excessive self-citations)- we apply a computational process to filter out suspicious links. **The core** of the solution is examining citation steadiness under perturbations. We randomly and controlledly remove a portion of the edges in the network and attempt to reconstruct them using learning algorithms (**GraphSAGE and Node2Vec**). The hypothesis is that genuine connections (citations) will be successfully and consistently reconstructed because they are supported by "indirect paths" and semantic/structural proximity that is preserved even after the perturbation. In contrast, manipulative connections, which do not fit into the natural scientific fabric, will repeatedly fail to be reconstructed.

**3.2. Impact Factor: Definition and Calculation-** Before presenting the algorithm, it is essential to understand the metric we aim to protect. The Impact Factor is a measure reflecting the yearly average number of citations to recent articles published in a specific journal. This metric directly influences the prestige of researchers and institutions and serves as a decisive factor in promotion and funding decisions, therefore it's a very important measure that must be reliable.

**Calculation Method:** For a specific year  $Y$ , the IF of a journal is calculated as follows:

$$IF_Y = \frac{\text{Citations in } Y \text{ to items published in } Y - 1 \text{ and } Y - 2}{\text{Total citable items published in } Y - 1 \text{ and } Y - 2}$$

In other words, the numerator is the total number of citations received in the current year to items published in the two previous years, and the denominator is the number of articles (citable items) published in those two years. In this project, we calculate the "Regular IF" based on raw data, and subsequently calculate an "Improved Impact Factor" based solely on edges identified as reliable by our algorithm. The difference between the two metrics reveals the extent to which unstable (suspected manipulative) citations influence the journal's ranking.

**3.3. The Main Algorithm Steps-** The workflow consists of a series of steps designed to quantify link reliability and compare two embedding methods. Below are the steps as defined in the main algorithm:

1. **Input and Data Preparation:** receives a citation graph  $G=(V,E)$  (treated as an undirected graph in order to focus on the structural rather than directional analysis) along with a feature matrix for each node. We initialize two result arrays of length equal to the number of edges in the graph, setting them to zero. These arrays will count how many times each edge was successfully reconstructed by each model.
2. **Time Loop:** for a pre-defined time frame, our algorithm will run in a loop where all the perturbations and calculations are done yearly. That's so the IF can be calculated yearly with the relevant updated graph.
3. **Perturbation Loop:** The process runs for a predefined number of iterations ( $N_{iter}$ ). In each iteration:
  - **Creating a Temporary Graph ( $G_T$ ):** Randomly remove a certain percentage ( $Fr$ ) of the edges from the original graph. This is the "perturbation" step designed to test network resilience.

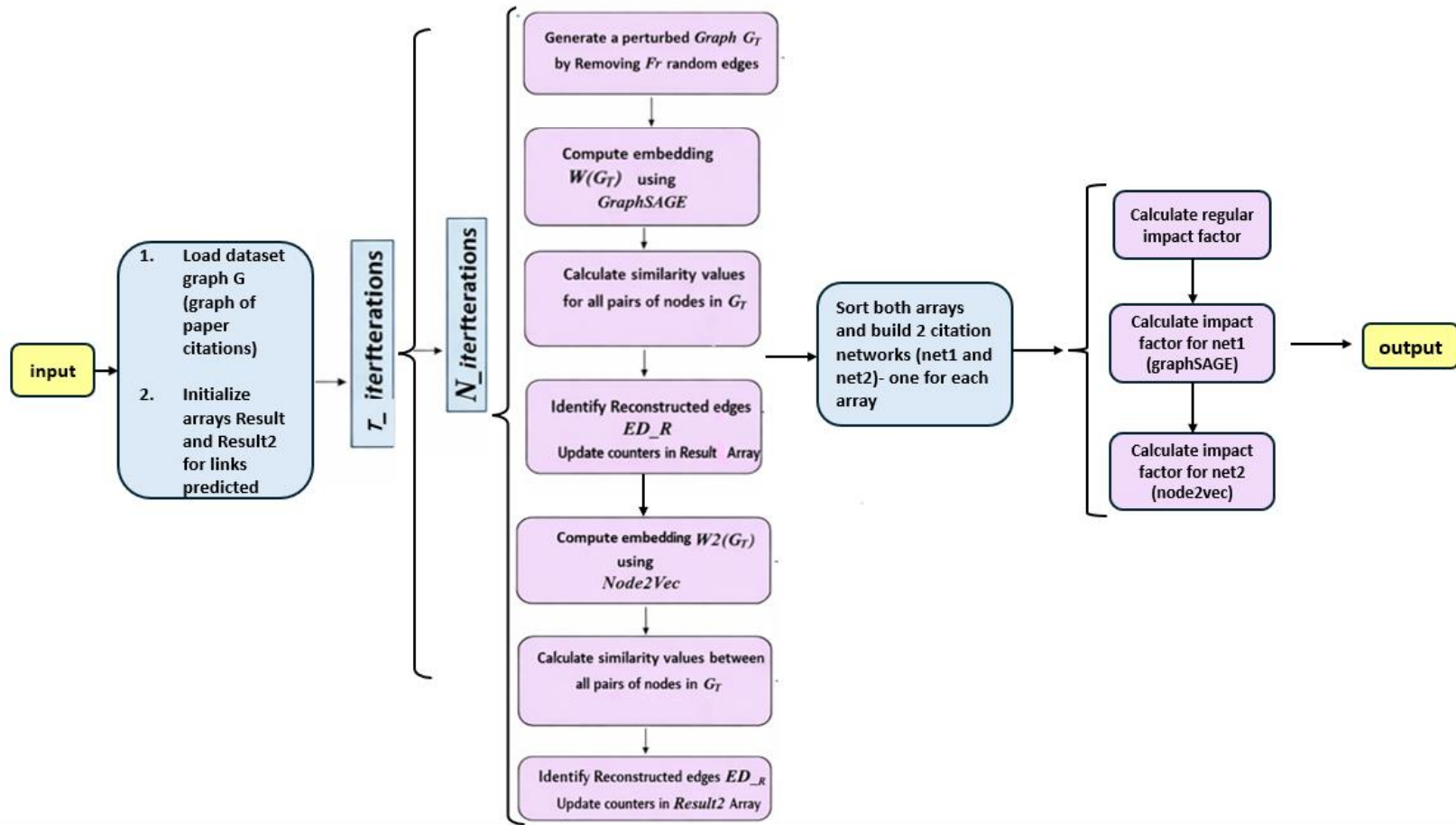
**Steps 3-4 happen in the perturbation loop-**

4. **Embedding Generation:** For the temporary graph  $G_T$ , we generate vector representations for nodes using two methods simultaneously for comparison:
  - **Model 1 - GraphSAGE:** An inductive method that learns to generate embeddings by sampling and aggregating information from neighbors. This method is effective in generalizing to evolving graphs.
  - **Model 2 - Node2Vec:** A method based on Random Walks in the graph and using the Skip-gram algorithm to map nodes with similar structural roles or local neighborhoods to close vectors.
5. **Link Prediction and Result Update:** For each model, we calculate the Similarity Score between every pair of nodes in the perturbed graph  $G_T$  using cosine similarity.
  - If the similarity score exceeds a defined threshold ( $Tr$ ), the algorithm predicts that an edge exists ("Edge Reconstruction").
  - For each successfully reconstructed edge, we update the corresponding counter in the result array (Result for GraphSAGE or Result2 for Node2Vec).
6. **Building the Reliable Network and Metric Calculation:** After completing the iterations, we sort the results. Edges with a low reconstruction score are considered unreliable and are filtered out. Finally, we perform a comparative calculation for two-year time windows:
  - Calculate the **Regular Impact Factor** (based on all data).
  - Calculate the **Improved Impact Factor** (according to GraphSAGE).
  - Calculate the **Improved Impact Factor** (according to Node2Vec).

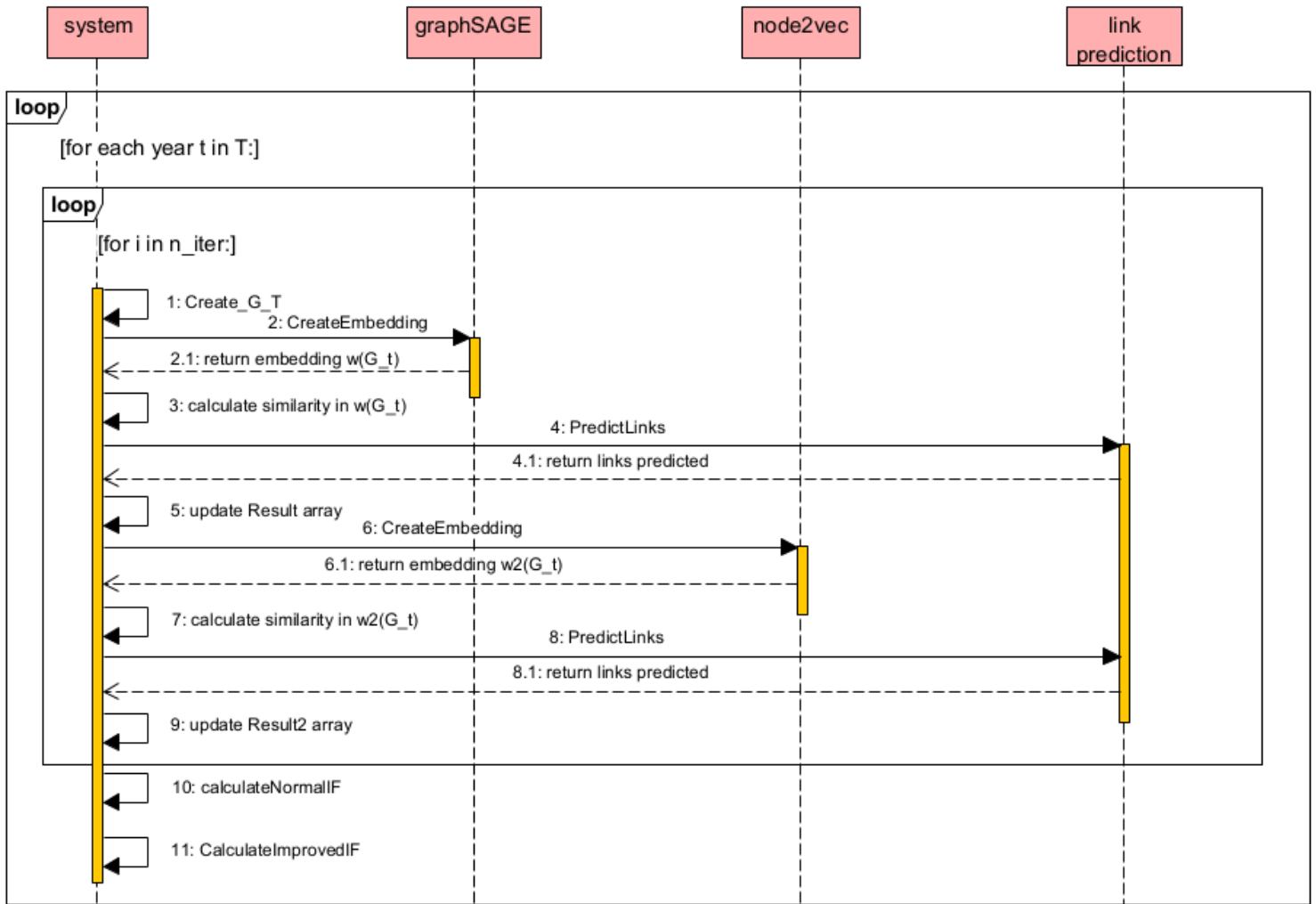
This comparison between all 3 impact factors allows us to present a more transparent tool for research evaluation, neutralizing the influence of noise and manipulations.

## 4. Diagrams

### 4.1 Suggested algorithm flow diagram-



#### 4.1 Suggested algorithm sequence diagram-



## 5. Testing plan

Relevant test we ran to make sure everything works as expected:

### 5.1 Result accuracy tests:

| Test                                                                                  | Input                                 | Result                                                                           |
|---------------------------------------------------------------------------------------|---------------------------------------|----------------------------------------------------------------------------------|
| Adding random edges to the graph synthetically to make sure they are removed          | The updated graph                     | The edges fail to reconstruct                                                    |
| Running the code with $n\_iter=\{10,50,100,200\}$ to make sure the results are stable | Different $n\_iter$ value in each run | Impact Factor values are very close (or equal) for every number or perturbations |
| Running the code with $Fr=\{0.3, 0.4, 0.5\}$ to make sure the results are stable      | Different $Fr$ value in each run      | Impact Factor values are very close (or equal) for every fraction value          |
| Compare the IF values for each year                                                   | All relevant parameters               | Normal IF value > modified IF values                                             |

### 5.2 Formal tests:

| Test                                                                                                            | Input                  | Result                                                                                       |
|-----------------------------------------------------------------------------------------------------------------|------------------------|----------------------------------------------------------------------------------------------|
| To make sure the data from the api is loaded correctly – load it twice and check that the data is the same size | Data api import result | in every import, the data size and structure are the same                                    |
| Fr removal correctness                                                                                          | graph, $Fr$ value      | The amount of edges in the graph is $ E -Fr( E )$ – meaning the correct fraction was removed |
| Embedding correctness                                                                                           | graph                  | All embedding vectors are the correct size, none is NULL                                     |
| Lwalk correctness – each walk is different than the other                                                       | graph                  | Each walk is different, none is repeated twice in the same iteration                         |

## 6. Results

In our experiments and tests, we have come to multiple important results.

### 6.1. Main result

The goal of this project was to test whether the perturbation-and-reconstruction algorithm can refine the Impact Factor by removing citations that are not supported by the surrounding network structure. Under the project's hypothesis, citations that fail to reconstruct after repeated perturbation are suspect or unreliable (reliability of this filtering is validated separately in Section 6.2), while genuine citations, anchored in the broader citation fabric, can be recovered.

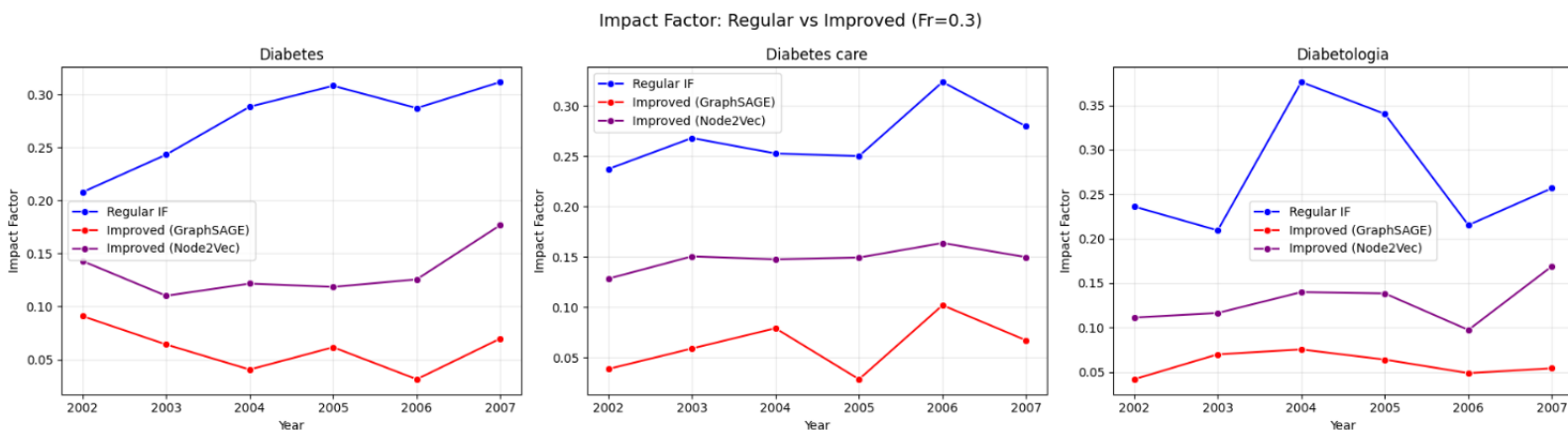
In every (journal, year) cell, comparing the improved IF with the regular IF showed that the Improved IF was lower than the Regular IF- which is the expected direction if it is removing unreliable links.

The magnitude of the reduction differed between the two embedding methods:

- **GraphSAGE** produced the more aggressive filtering. At its operating threshold ( $Tr = 0.95$ ), the Improved IF fell to roughly one-fifth to one-quarter of the Regular IF.
- **Node2Vec** was more conservative in what it removed, keeping the Improved IF closer to the original- roughly one-half of the Regular IF.

The fact that two structurally different embedding techniques produced a consistent reduction in the IF indicates that the suggests the effect is related to the underlying structure of the citation graph: a substantial fraction of the raw citations are not robustly supported by their network neighborhood, and the refined metric discounts them.

Figure 4 shows the comparison of the Regular Impact Factor with the improved Imoact Factor (using both embedding methods) across several years for the 3 most common journals in the dataset. In all cases, the improved IF values remains consistently below the regular IF.



**Figure 4:** Comparison graph of improved IF values vs regular IF values

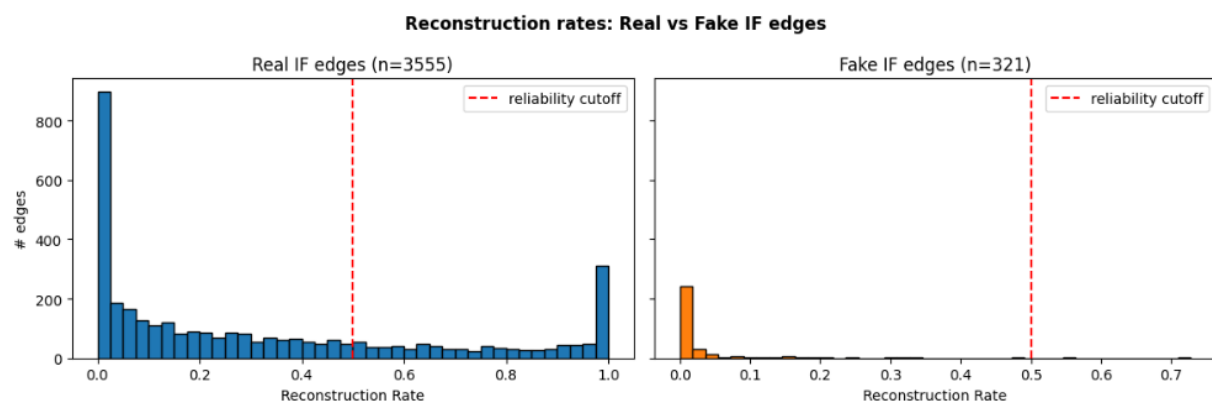
This produces the project's intended output: a more conservative Impact Factor that reflects only the citations that remain structurally supported, offering a complementary, more conservative alternative to the raw metric.

## 6.2. The fake-edge control

For each year, we have injected 10% fake edges to the graph, to test its behavior and check its compatibility with our initial hypothesis.

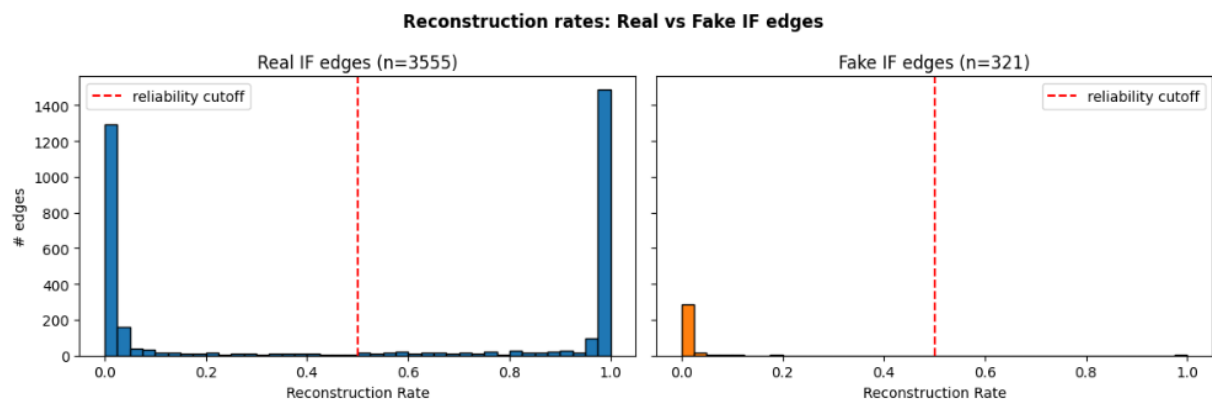
The results were as follows:

GraphSAGE: out of all the fake edges injections, only 0.6% survived with a reconstruction rate of over 0.5 after all the perturbations, versus 28.2% of the original edges in the graph.



**Figure 5:** Comparison graph of reconstruction rates between the original and the fake edges added artificially for GraphSAGE

Node2vec: out of all the fake edges injections, only 1.6% survived with a reconstruction rate of over 0.5 after all the perturbations, versus 52.8% of the original edges in the graph.



**Figure 6:** Comparison graph of reconstruction rates between the original and the fake edges added artificially for Node2vec



The conclusion: we can conclude that the code is running according to the hypothesis, and the filtering process is not random; the algorithm rejects fabricated citations more aggressively than authentic ones.

### 6.3. Fr-invariance

For both models, running on different FR values only made a slight difference in the results. The reason is structural: the perturbations only happen for the edges that are relevant to the IF calculation. Therefore, even  $Fr=0.7$  perturbs only a small fraction compared to the full training graph.

This comes to demonstrate that authentic edges are recoverable whether you remove 30% or 70% of them, because their support comes from the broader neighborhood, not from any single removed edge.

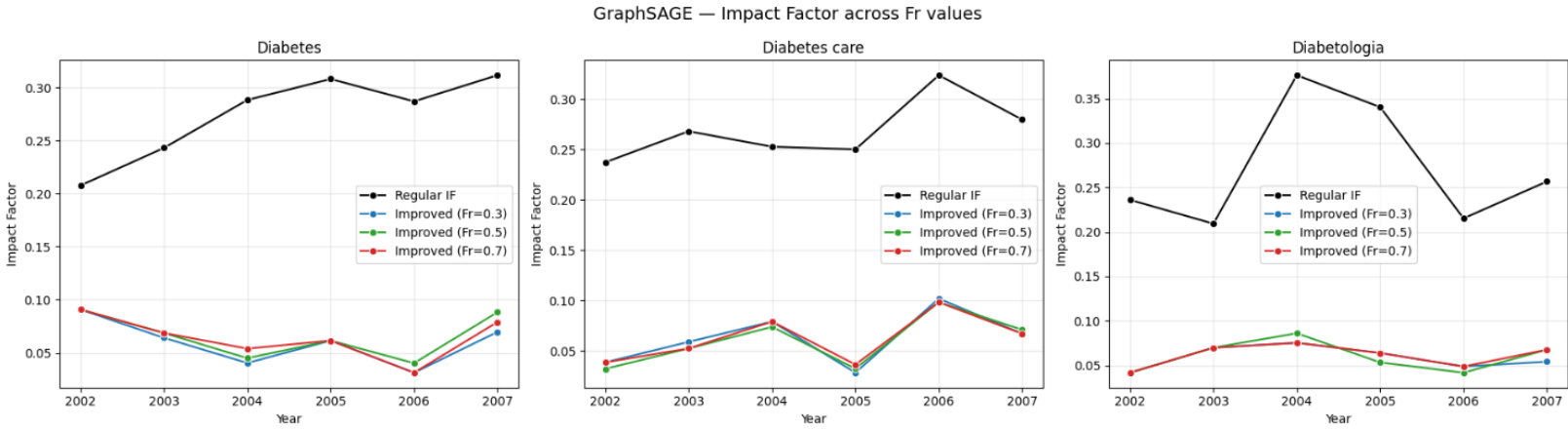


Figure 7: IF result differences for  $Fr=\{0.3,0.5,0.7\}$  for the most common journals in the dataset, using GraphSAGE

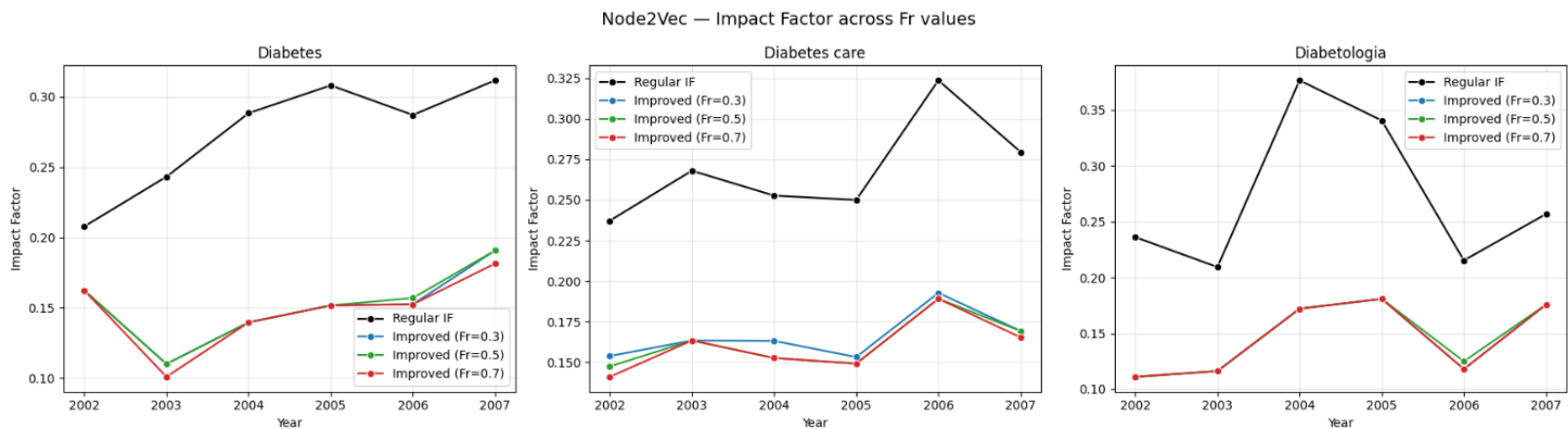


Figure 8: IF result differences for  $Fr=\{0.3,0.5,0.7\}$  for the most common journals in the dataset, using Node2vec

## 6.4. Thresholds sensitivity

In our diagnostics, we concluded that there's a difference between the clustering of the removed-edge cosine similarities: GraphSAGE cluster around 0.85-0.9, while Node2vec cluster around 0.3-0.35. This means the steadiness criterion is not a universal cosine number- it's model-relative.

To check how meaningful the threshold value is, we decided to run it for "permissive → moderate → strict" values:

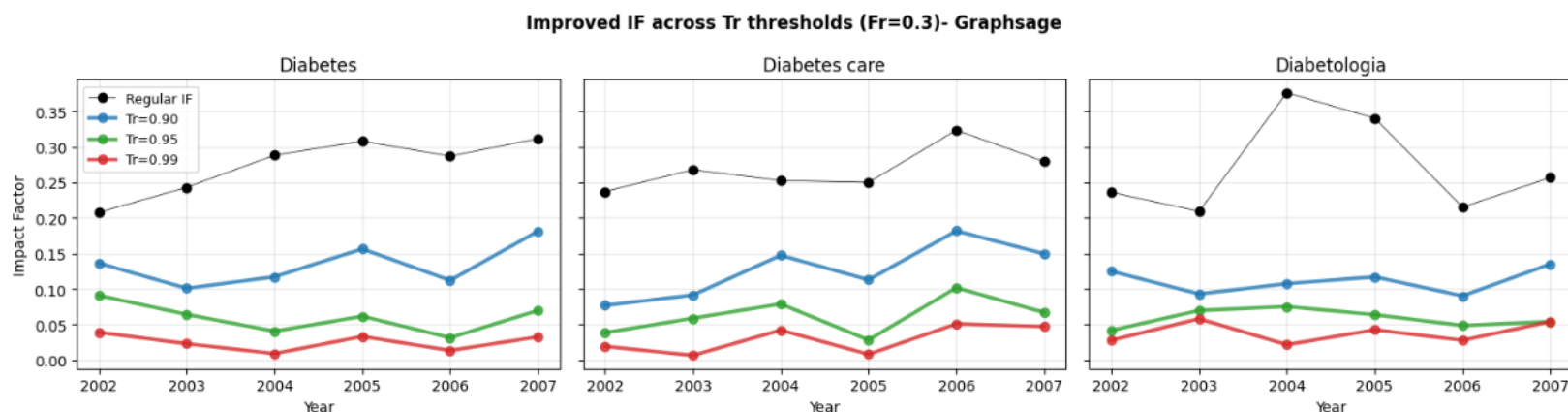
Permissive= approximately the median value. GraphSAGE- 0.9, Node2vec- 0.3

Moderate= just above median. GraphSAGE- 0.95, Node2vec- 0.4

Strict= clearly above median. GraphSAGE- 0.99, Node2vec- 0.5

As expected, the results showed that looser threshold → more edges clear the reliability cutoff → Improved IF rises toward Regular IF; stricter threshold → fewer edges survive → Improved IF drops.

The conclusion: the contrast with Fr shows, that **Fr have very little influence on the results, while the Tr is the one that actually governs the outcome**- and it must be set relative to each embedding's similarity distribution.



**Figure 9:** IF result differences for TR={0.9,0.95,0.99} for the most common journals in the dataset, using GraphSAGE

Improved IF across Tr thresholds (Fr=0.3) — Node2Vec

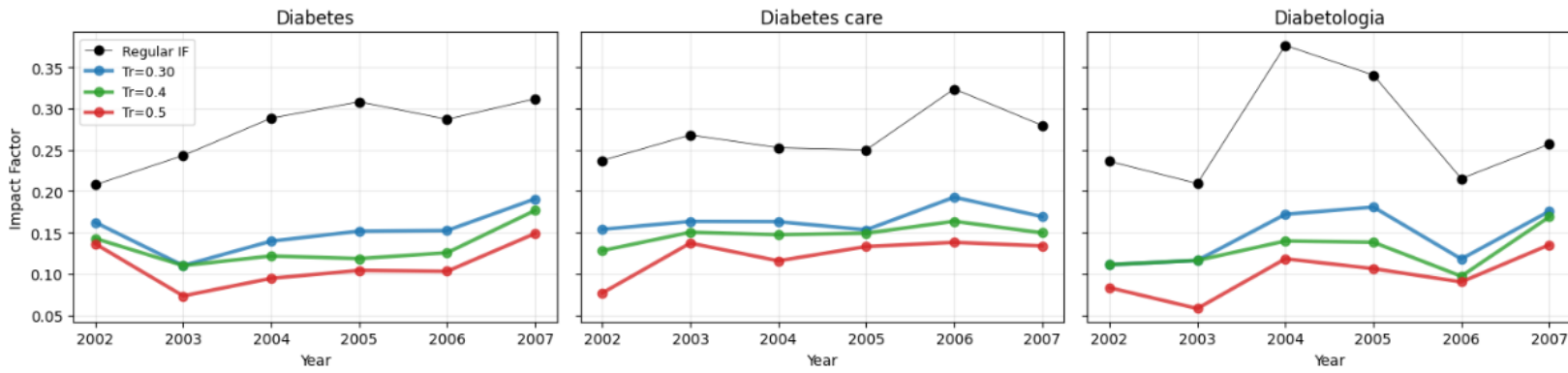


Figure 10: IF result differences for TR={0.3,0.4,0.5} for the most common journals in the dataset, using Node2vec

## 7. Conclusions

Under the steadiness hypothesis of [2][3], an edge that repeatedly fails reconstruction is treated as potentially manipulated.

On the PubMed diabetes network, both GraphSAGE and Node2Vec independently reduce the Improved IF below the Regular IF in every configuration, but to different degrees: under our parameters settings, GraphSAGE is the more aggressive filter, classifying around 3/4 to 4/5 of raw citations as structurally unsupported and could be manipulative (Improved IF roughly a quarter of Regular IF or lower), while Node2Vec is more permissive and classifies around 1/2 of citations as structurally unsupported/manipulative (Improved IF roughly half to two-thirds of Regular IF).

A fake-edge control confirms this filtering is genuine rather than indiscriminate: synthetic (manipulation-like) citations survive the reliability cutoff 36-47× less often than authentic ones (0.6% vs 28.2%, 1.6% vs 52.8%).

The result is invariant to the perturbation fraction  $F_r$  but mainly determined by the reconstruction threshold  $T_r$ , which must be calibrated to each embedding's similarity distribution.

## 8. Summary

This project addressed a core weakness in one of academia's most influential metrics: the Impact Factor is computed on raw citation data, with no mechanism to filter out manipulation. We set out to produce a more trustworthy alternative — an "Improved Impact Factor" - by keeping only the citations that the network's own structure can corroborate.

The experiments on our implementation of the algorithm showed the algorithm works as intended. Both embedding methods produced a consistent, downward correction of the Impact Factor across every journal and year, indicating that a substantial portion of raw citations are not robustly supported by the surrounding network. A dedicated fake-edge control confirmed that this filtering is meaningful rather than random: artificially injected citations were rejected far more aggressively than genuine ones.

Taken together, these results demonstrate that structural reconstruction can serve as a practical filter for citation reliability, yielding a more conservative and skeptical Impact Factor that complements the standard metric. Future work could extend the approach to coordinated manipulation patterns such as citation cartels- where fabricated edges share structural neighborhoods and are harder to detect than the random injections tested here- and to additional datasets and embedding methods beyond the two compared in this project.

## References

- [1] Avros, R., Farfel, G., & Volkovich, Z. (2025). *A Nested Structure of Anomalies in Academic Publication Citations*. In Proceedings of the 14th International Conference on Data Science, Technology and Applications (DATA 2025), pages 761-768.
- [2] Avros, R., Toledano Kitai, D., & Volkovich, Z. (2025). *Citation Steadiness Analysis with GraphSAGE Approach*. Proceedings of the 14th International Conference on Data Science, Technology and Applications (DATA 2025), pages 769-777.
- [3] Avros, R., Keshet, S., Toledano Kitai, D., Vexler, E., & Volkovich, Z. (2023). *Detecting Pseudo-Manipulated Citations in Scientific Literature through Perturbations of the Citation Graph*. Mathematics, 11, 3820.
- [4] Grover, A.; Leskovec, J. *Node2vec: Scalable Feature Learning for Networks*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '16, San Francisco, CA, USA, 13–17 August 2016; pp. 855–864.
- [5] Hamilton, W. L., Ying, Z., & Leskovec, J. (2017). *Inductive representation learning on large graphs*. Advances in Neural Information Processing Systems (Vol. 30). Curran Associates, Inc.
- [6] Prabha, C. G. (1983). *Some aspects of citation behaviour: A pilot study in business administration*. Journal of the American Society for Information Science, 34(4), 202-206